STUDENT

0003-RAO

TENTAMEN

220826_DV2606_2110_Salstentamen

Kurskod	--
Bedömningsform	--
Starttid	26.08.2022 09:00
Sluttid	26.08.2022 14:00
Bedömningsfrist	--
PDF skapad	21.05.2024 13:16
Skapad av	Zandra Johansson

1 There are, in general, two approaches to communicate between the processors in a MIMD parallel computer: *shared-memory* and *message-passing*.

- Describe the main characteristics of each of the two approaches.
- Further, present some advantage and disadvantage for each of them.
- Finally, give some example of parallel applications (with motivation) where shared-memory and message-passing would be preferable over the other, respectively.

Shared-memory

A shared memory computer facilitates communication between processors through a global shared memory. The memory may be physically shared or distributed with software managing access to the distributed memory so it is accessible to all processors.

An advantage is that processor cooperation on large datasets is simpler as the data is accessible to all processors and communication between processors may not be needed for more than work division.

This means that it is useful for parallelized sorting workloads that can be split up into smaller independent chunks that don't require communication between processors. Since the processors are working directly in shared memory it also means that all other processors have immediate access to the updated data.

This is both and advantage and a disadvantage as atomic write operations to the output datastructure can cause processors to idle waiting for their turn.

Message-passing

A message-passing computer has to explicitly send and receive messages between processors.

Since there is no shared memory, a main processor has to send the workload to the other processors either initially (though the workload may be too large to fit in the local memory of the processors) or through a continuous stream. A disadvantage here is that it can cause a lot of communication on the bus, which could itself cause idling.

Ord: 216

Besvarad.

2 When we parallelize a sequential application, e.g., by using threads or message-passing, so it can can execute in parallel on *N* processors, we often find that the parallel program is not *N* times faster. Describe two important reasons for that.

Parallelizing an application will always introduce some amount of overhead for dividing workloads, synchronizing threads, etc. This means that just because the parallel version may be faster, it can perform more computations in total. Due to this is cannot be N times faster.

Another reason is that for a lot of programs it may not be possible to parallelize a large part of it. If only 90% can be parallelized, even if it's done perfectly with an infinite amount of processors, 10% will still be done sequentially. This limits the program to a speedup of 100/10=10.

Ord: 96

Besvarad.

3 Sometimes we find that a system or application has a *superlinear speedup*. What does that mean? Describe two situations when it may occur.

Superlinear speedup means that a parallelized program running on N processors is more than N times faster than the sequential counterpart.

One reason for this may be that the parallel program does less work than the sequential one.
For example when searching a binary tree. The parallel version can look in multiple branches at once and finds the goal node in less total iterations.
If we have 3 branches and use depth first left to right search, and the goal node is 3 nodes down the middle branch. While the sequential program goes down the entirety of the left-most branch, the parallel version goes down three nodes in all branches and then finds the goal node.

Another reason is due to cache effects. If a 1mb block is swapped into cache, but data from two different 1mb blocks are needed, they will be swapped multiple times causing downtime. If two processors are used that can hold 1mb block each, then no swapping is needed and computations can be run continuously.

Ord: 170

Besvarad.

4 *Cache coherence* is an important issue to handle in a shared-memory multiprocessor. Describe, by giving an example, why ensuring cache coherence is important. (1p)

Two main approaches exist to maintain cache coherence: *write-invalidate* and *write-update*. Briefly describe each of them, e.g., by drawing a simple picture. (2p)

Describe some advantage and disadvantage with each of the approaches. (1p)

Cache coherence ensure that all instances of a value is always updated to the most recent one. This is important as many applications rely on the result of other processing units.

Write-invalidate

Let's say we have two processors P_1 and P_2.

When P_1 performs a write on a memory block, the cache controller broadcasts the address of the memory block on the bus allowing the cache controller of P_2 to mark the block as invalid in its cache. When P_2 tries to read a value in the memory block, the cache controller sees it is is invalidated and updates the block from the most recent source.

A disadvantage is that P_2 has to idle while the updated block is fetched.

An advantage is that a lower bandwidth bus can be used even for large amounts of processors, resulting in much greater scalability.

Write-update

Let's again say we have two processors P_1 and P_2.

When P_1 performs a write on a memory block, the cache controller broadcasts the address of the memory block on the bus allowing the cache controller of P_2 to immediately update it from the most recent source. When P_2 tries to read a value in the memory block it is then immediately available and can be used and no downtime occurs, which is a nice advantage. A huge disadvantage is however that this creates a lot of traffic on the bus and isn't very scalable as a very high bandwidth bus is required for many processors.

Ord: 249

Besvarad.

5 When writing a parallel application, we can use mainly two approaches: *task (a.k.a. functional) parallelism* and *data parallelism*.

- Describe the main characteristics of each of the two approaches.
- Further, describe some advantage and disadvantage with each of them.
- Finally, give some examples of applications (with motivation) that are suitable for task parallelism and data parallelism, respectively.

Task/Functional Parallelism

This approach identifies the independent tasks of an application and tries to achieve a speedup by dividing these tasks up into threads.

An advantage is that it can be easy for the programmer to split up the workload like this.

A disadvantage is that is can be difficult to balance the workload between processors as tasks are rarely equal.

Task parallelism makes more sense to use for applications that have multiple 'tasks' being done. A task pool can even be created where processors can fetch tasks that need to be done and execute them. The goal would be to allow tasks that don't depend on each other to run in parallel, resulting in a speedup.

Data Parallelism

Data parallelism identifies how data processing can be split up into independent chunks.

The advantage here is that the programmer has much finer control over how the workload is balanced between the processors, generally resulting in greater speedup.

A disadvantage is that it can be more time consuming to identify how the data can be split and what the best way to divide the workload would be.

Most applications that perform a lot of data processing, such as gaussian elimination, would benefit from data parallelism as the compute-heavy parts can often be divided up into separate workloads.

Ord: 216

Besvarad.

6 The *PRAM* (*Parallel Random Access Machine*) model has often been criticized for being unrealistic. Why?

Why, and in which situations/contexts, does it still make sense to consider it?

It is generally considered unrealistic as it assumes that memory accesses can be done in unit time anywhere in memory which is almost never the case.

It is still very useful as a best case scenario when evaluating the efficiency of parallel algorithms.

Ord: 43

Besvarad.

7 Consider the following (pseudo) code segments with critical sections.
Initially, we assume initially that:

```
bool flag1 = false;
bool flag2 = false;
pthread_mutex_t mtx = unlocked;
```

Thread 1:

```
/// do something
pthread_mutex_lock(&mtx)
    while (!flag1);
    flag2 = true;
pthread_mutex_unlock(&mtx)
/// do something more
```

Thread 2:

```
/// do something
pthread_mutex_lock(&mtx)
    while (!flag2);
    flag1 = true;
pthread_mutex_unlock(&mtx)
/// do something more
```

Is this code correct or not? Clearly motivate why / why not!

The code would cause deadlocks as they would both try to obtain the lock and then wait for the other thread to set their flag for them to continue and set the others flag.

Let's say thread 1 obtains the mutex.
This means that thread 2 sits at pthread_mutex_lock(&mtx) and waits for it to be released by thread 1.
Thread one is waiting for thread 2 to set flag1 to true. Both threads are waiting for eachother, resulting in a deadlock.

It could have been thread 2 that obtained the mutex, and the waiting points would be reversed. Either way would result in a deadlock.

```
bool flag1 = false;
bool flag2 = false;
pthread_mutex_t mtx = unlocked;
```

Thread 1:

```
/// do something
pthread_mutex_lock(&mtx)
    while (!flag1);   <--Thread 1 is stuck here waiting for thread 2 to set flag1 = true;
    flag2 = true;
pthread_mutex_unlock(&mtx)
/// do something more
```

Thread 2:

```
/// do something
pthread_mutex_lock(&mtx)  <-- Thread 2 is stuck here waiting for thread 1 to realease the mutex.
```

```
    while (!flag2);
    flag1 = true;
pthread_mutex_unlock(&mtx)
/// do something more
```

Ord: 172

Besvarad.

8 Consider the sequential C code below for performing matrix-vector multiplication.

```
1  #define SIZE 4096
2  static double a[SIZE][SIZE], x[SIZE], y[SIZE];
3
4  static void init_values(void)
5  {
6      ... // init matrix a and vector x
7  }
8
9  static void matvec_seq(void)
10 {
11   int i, j;
12   for (i = 0; i < SIZE; i++) {
13     y[i] = 0.0;
14     for (j = 0; j < SIZE; j++)
15       y[i] = y[i] + a[i][j] * x[j];
16   }
17 }
18
19 int main(int argc, char **argv)
20 {
21   init_values();
22   matvec_seq();
23 }
```

Write (in C-like pseudo code) a parallel version of the matrix-vector multiplication algorithm, i.e., the function *matvec_seq()*, using a *1-dimensional partitioning*. Clearly state which assumptions you do about the type of machine and programming model. Assume that the number of processors is significantly fewer than SIZE. (2p)

Analyze your solution from a performance point of view, e.g., approximate speed-up, load-balancing, etc. (2p)

Assuming that a shared memory computer is used.

```
#define SIZE 4096
#define NUM_OF_CPUS 8
static double a[SIZE][SIZE], x[SIZE], y[SIZE];

static void init_values(void)
{
 //init a and x
}

static void matvec_par()
{
    pthread threads[NUM_OF_CPUS-1]
   //Launch worker threads
   for (int i = 1; i < NUM_OF_CPUS; i++)
      {
         threads[i] = pthread_create(matvec_par_thread(i));
      }
```

```
    //Do some work in the main thread
    //Using a stride to attempt to benefit from memory bursting
    for (i = 0; i < SIZE; i = i + stride)
    {
        y[i] = 0.0;
        for (j = 0; j < SIZE; j++)
        {
            y[i] = y[i] + a[i][j] * x[j];
        }
    }

    for (int i = 1; i < NUM_OF_CPUS; i++)
        {
            pthread_join(threads[i]);
        }
}

static void matvec_par_thread(int thread_id)
{
    int i, j;
    int stride = NUM_OF_CPUS;
    //Using a stride to attempt to benefit from memory bursting
    //Note that thread_id starts at 1, some work is done in the main thread
    for (i = thread_id; i < SIZE; i = i + stride)
    {
        y[i] = 0.0;
        for (j = 0; j < SIZE; j++)
        {
            y[i] = y[i] + a[i][j] * x[j];
        }
    }
}

int main(int argc, char **argv)
{
    init_values();
    matvec_par();
}
```

Ord: 195

Besvarad.

9

Consider a sequence of 100 000 000 numbers that is stored consecutively in global memory (i.e., stored one number after another in memory). You would like to count the number of even and the number of odd numbers in this sequence and use 10 000 CUDA threads to do this job (i.e., each thread should process 10 000 numbers). You would also like to benefit from memory bursting. The way you distribute the different numbers among the threads affects the memory access pattern. Suggest one way to distribute the work among the threads that benefits from memory bursting and one way of distributing the work that does not benefit from memory bursting. Also explain why one solution benefits from memory busting and the other solution does not.

Benefitting from memory bursting

Using a stride like this:

P_1	P_2	P_3	P_4	P_5	P_1	P_2	P_3	P_4	P_5	P_1	P_2	P_3	P_4	P_5	P_1	P_2	P_3	P_4	P_5

to always access nearby memory locations would be utilizing the DRAM memory bursting. This would happen since the DRAM sends surrounding bits as well no matter if they are used or not, unused ones would simply be discarded. If a stride is used to ensure they are used, less memory accesses would need to be done.

No memory bursting

Simply dividing an array up into equal chunks like this:

P_1	P_1	P_1	P_1	P_2	P_2	P_2	P_2	P_3	P_3	P_3	P_3	P_4	P_4	P_4	P_4	P_5	P_5	P_5	P_5

Would not benefit from memory bursting as memory accesses would ben distributed in memory, resulting in a lot of the bits sent by the dram to simply be discarded.

Ord: 147

Besvarad.

10 Thread Blocks and Warps

Select one or more alternatives. Selecting an alternative that is incorrect will give a negative score. The minimum score on this assignment is 0.

- [ ] Threads from different Thread Blocks can execute in the same Warp.
- [ ] Threads in the same Warp can execute on different Streaming Multiprocessors.
- [x] A Thread Block can contain many Warps.
- [x] Threads in the same Warp cannot execute different instructions at the same time.
- [ ] A Thread Block cannot be larger than a Warp.

Rätt. 2 av 2 poäng.

11

Select one or more alternatives. Selecting an alternative that is incorrect will give a negative score. The minimum score on this assignment is 0.

- [ ] The processor cores in a CPU normally executes in SIMD mode.
- [ ] A CPU normally has more processor cores than a GPU
- [x] A CPU normally has larger caches than a GPU
- [ ] The processor cores on a CPU are normally organized in streaming multiprocessors.
- [ ] Context switching on a GPU is normally faster than context switching on a CPU.

Delvis rätt. 1 av 2 poäng.

12

Select one or more alternatives. Selecting an alternative that is incorrect will give a negative score. The minimum score on this assignment is 0.

- [ ] Accesses to global memory are faster than accesses to shared memory.
- [x] Threads in different thread blocks cannot access the same variable in shared memor
- [ ] The CPU can access variables in shared memory
- [ ] All the threads in a thread block see the same registers
- [x] The content of the CUDA global memory is not changed between two kernel launche in the same program

Rätt. 2 av 2 poäng.

13

How can the programmer make the GPU put a variable in the cache? Where is such a variable stored when it is not in the cache?

Variables not stored in the cache are stored in the GPU registers.

Ord: 12

Besvarad.

14

What is thread divergence and why would we like to avoid thread divergence? Give an example of how thread divergence can be avoided or reduced.

Thread divergence happens when threads have branching statements that cause them to execute different instructions. The threads then execute different branches. This happens when a branching statement is based on the threadId. This should generally be avoided to reduce thread divergence.

Ord: 41

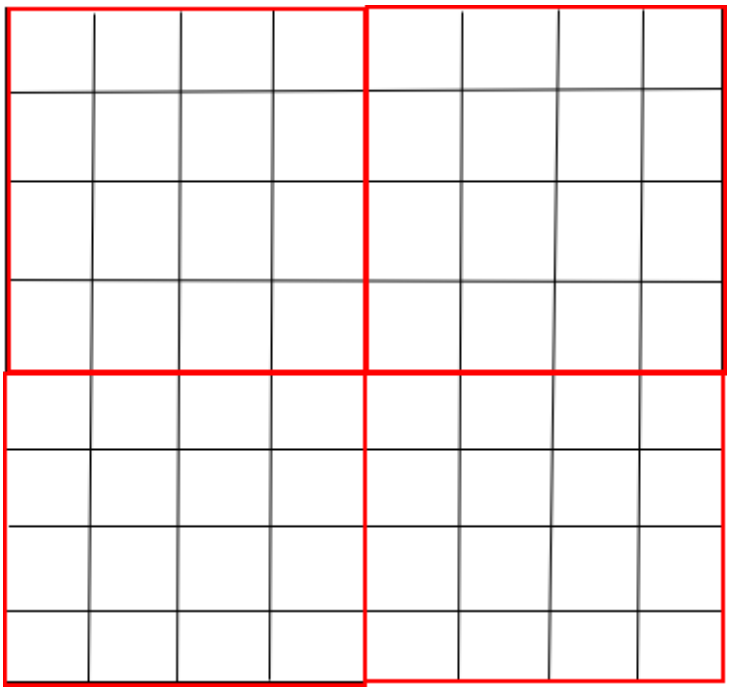
Besvarad.

15 Tiling

Tiling is a technique that can be used for improving the performance of some CUDA programs, e.g., programs that perform parallel matrix multiplication or convolution. Explain how tiling works and why tiling can improve performance. Also, explain why and how we may need to use __synchthreads() when using tiling (give a short example).

Tiling is used to divide a problem into smaller chunks that fit in the faster (but smaller) shared memory of the GPU instead of the smaller global memory. This can result in a speedup as memory accesses are much faster.

The 8x8 matrix below can be split into 4 individual 4x4 matrixes instead, and then be swapped into shared memory when work is done on them. __synctreads() may be used to know that all threads have finished working on the tile and are ready to move on to the next one. Swapping the tile too soon would mean that work isn't finished or incorrect data could be saved to the new tile. Both resulting in garbage data.



Ord: 117

Besvarad.

16 Privatization

Privatization is a technique that can be used for improving the performance of some CUDA programs, e.g., programs that perform parallel histogramming. Explain how privatization works and why privatization can improve performance.

Privatization is a method where a local copy of the output data structure is created. A processor would then write the results of its computations to the local copy instead of waiting for its result to be for written to the global version.

For write heavy applications this can reduce down time as atomic writes are done locally in uncontended memory, letting the processor get back to work faster. However, it may require calculating all the results together in the end.

Ord: 81

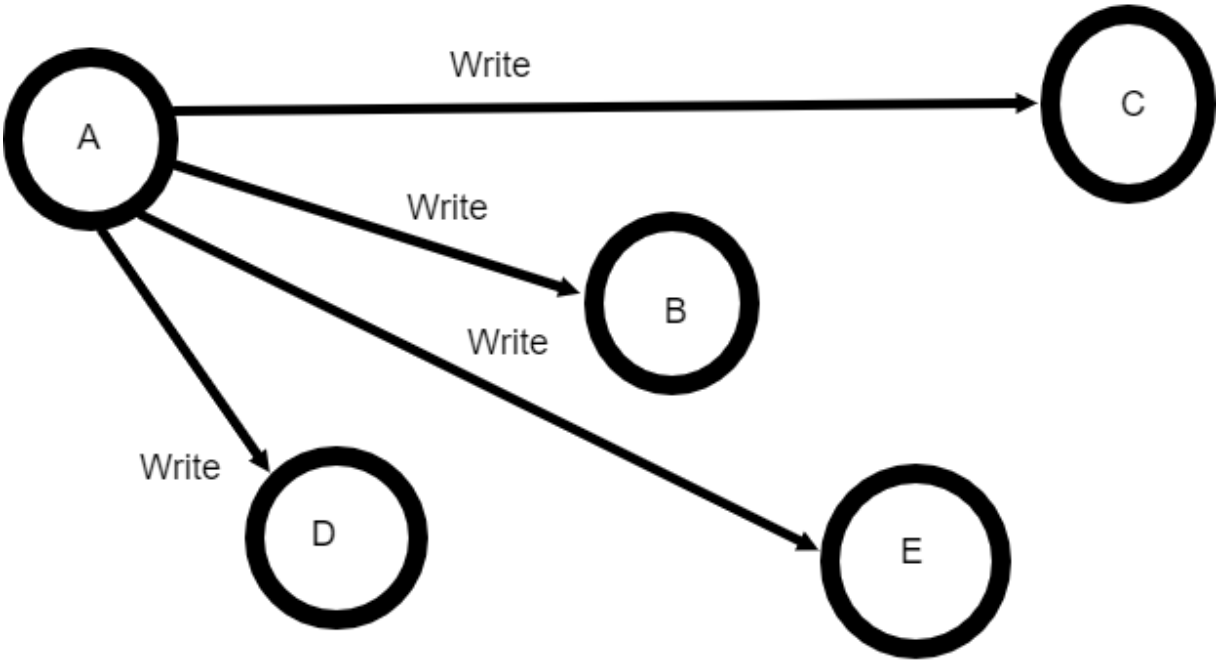
Besvarad.

17 Scatter to Gather

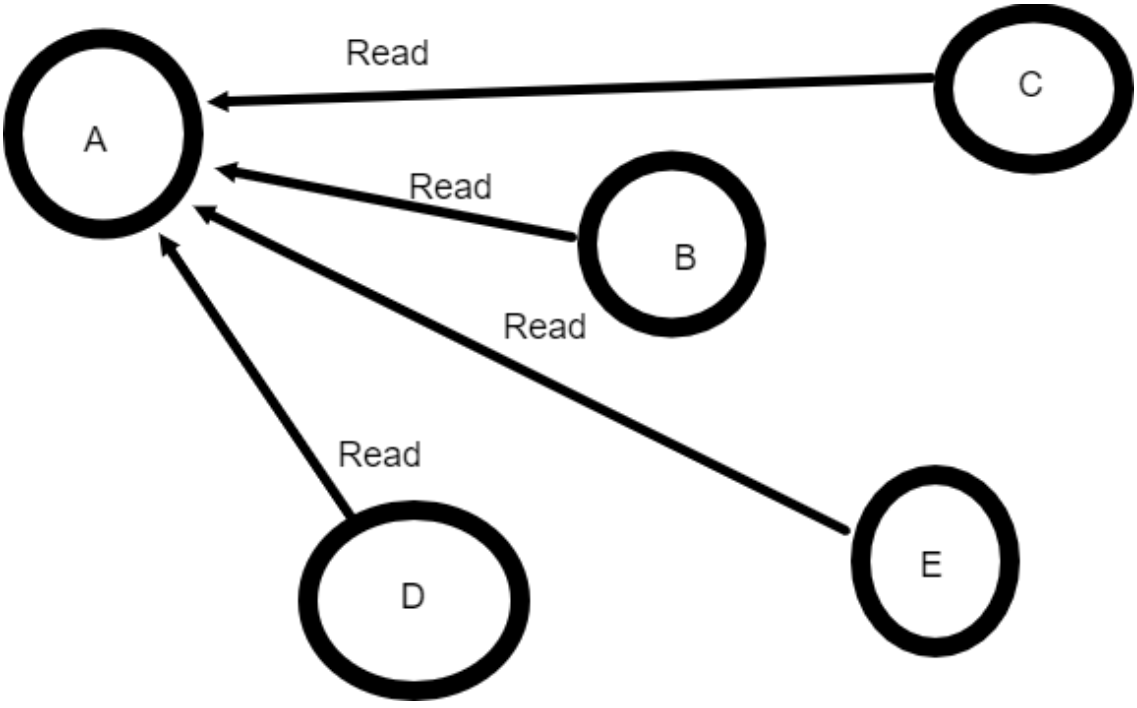
Scatter to gather conversion is a technique that can be used for improving the performance of some CUDA program. Explain how scatter to gather conversion works and why this technique can improve performance.

A scatter to gather conversion is done to reduce the number of atomic write operations.

Let's say we have 5 nodes, an example of a scatter operation would be for A to calculate how it affects all the other nodes and then try to write the result to them. When all the nodes try to do this all wat once, their memory is contended by many atomic write operations.



Converting this to a gather operations can be for node A to read information from the other nodes, calculate how it is affected by the other nodes, and the write the result only to its own memory.



This would divide the number of atomic write operations by 5.

Ord: 117

Besvarad.